

Moisture and Owned Buses

Observe a calibrated sample; preserve it as a bounded record

A potentiometer supplies repeatable moisture evidence while recording drivers isolate ownership from the landed Mega bus backends.

Lesson	022 — component	Time	150–210 minutes
Prerequisites	Lessons 001–021	Board	Arduino Mega 2560
Primary evidence	TP-M/A0, D13 pulse, D12 fault	Status	host verified; bench open
Energy class	E1: USB and resistor-limited LEDs only		

Scope. This lesson exercises the current pure `I2cBus`, `I2cDevice`, `SpiBus`, and `SpiDevice` interfaces with sketch-local recording drivers. Every transfer receives a bounded timeout value. The lesson generates no SDA, SCL, MOSI, MISO, or SCK waveform and communicates with no physical device. `MegaI2cDriver`, `MegaSpiDriver`, and their direct AVR backends now own the fixed pins and restore chip select in deterministic tests. This recording-driver sketch deliberately keeps their physical bench evidence open.

1 Mission

Build a repeatable transaction story:

TP-M → `MoistureSample` → bounded record → visible acknowledgement.

Turn a 10 kΩ potentiometer to create a controlled analog voltage. `MoistureSensor` maps the configured dry and wet readings to 0–1000 permille while keeping validity separate from the number. `SimulatedRtc` supplies explicit clock state, a synthetic I2C device exercises owned addressing, and a SPI recording transaction carries the same 11-byte record into `FixedStorage`. Only a successful `sync()` advances its durable prefix.

The exercise separates five ideas that are often hidden behind global `Wire` and `SPI` objects:

1. a physical observation and its interpreted sample are distinct;
2. a bus owner has one explicit lifetime;
3. a transaction has bounded, caller-owned buffers;
4. a driver performs transport but does not decide application policy;
5. an observation must say exactly what it proves.

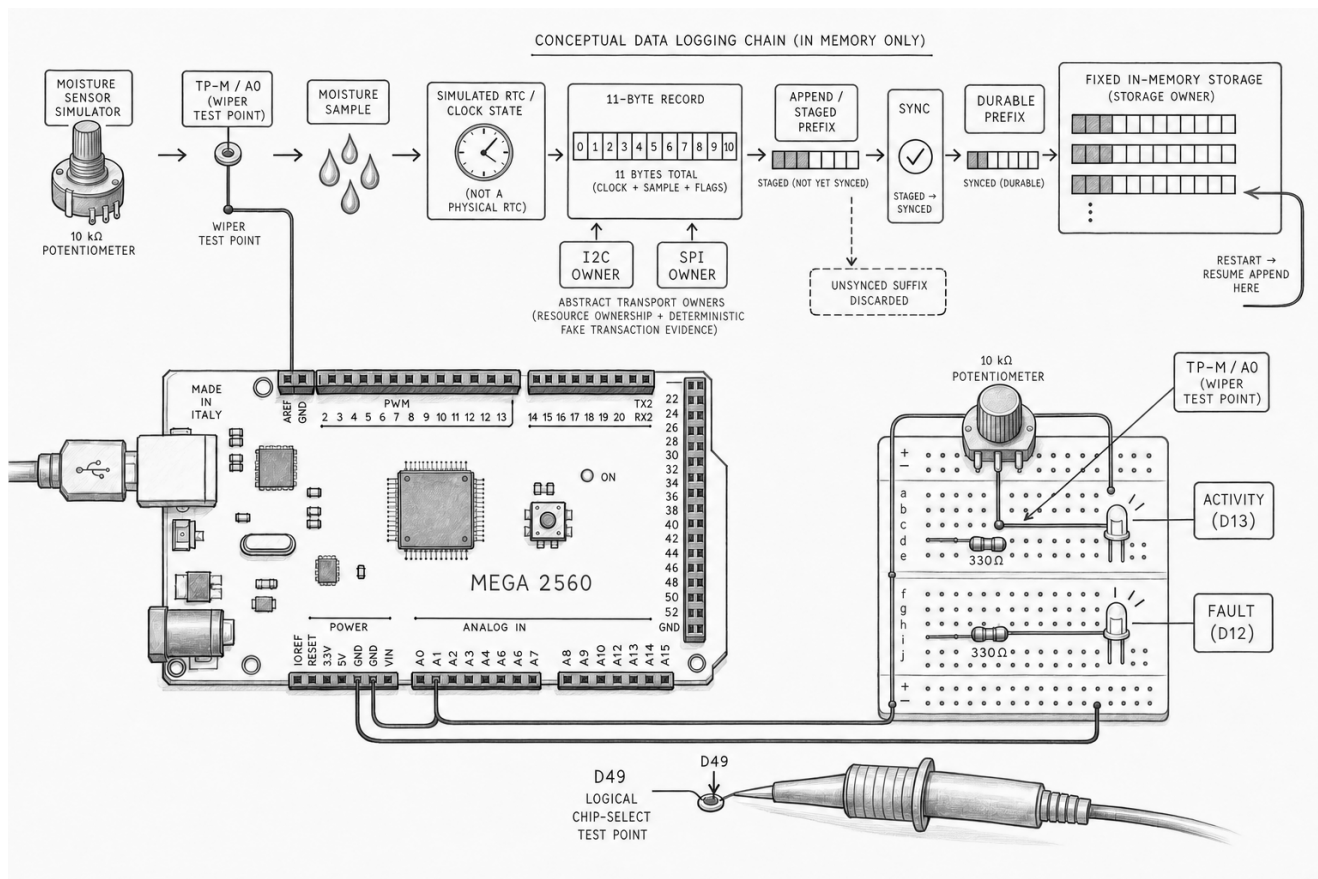
D13 pulses after a valid moisture sample receives a valid clock reading, passes both recording drivers, appends, and synchronizes. Its duration ranges from 100 ms at 0 permille to 900 ms at 1000 permille. D12 latches on an invalid sample, invalid clock state, transport failure, append failure, sync failure, or capacity exhaustion. The pulse proves the deterministic software path and in-memory durability model. TP-M is the independent electrical observation; neither proves a physical bus transaction or durable SD media.

2 Parts, limits, and wiring

Part	Requirement
Arduino Mega 2560	USB-powered board
Moisture simulator	10 k Ω linear potentiometer
Fault LED	external LED with one 1 k Ω series resistor
Activity LED	built-in D13 LED; no external connection
Optional meter	D12 safe-state check relative to a named GND point

Signal	Mega pin	Lesson connection
Moisture input / TP-M	A0 / D54	potentiometer wiper
Potentiometer rails	5 V, GND	one outer terminal to each rail
Activity evidence	D13	built-in LED; 100–900 ms accepted-sample pulse
Fault evidence	D12	D12 $\rightarrow$ 1 k Ω $\rightarrow$ LED anode; cathode $\rightarrow$ GND
Logical chip select	D49	resource identifier only; leave physically open
SDA/SCL	D20/D21	leave physically open; no waveform is generated
MISO/MOSI/SCK	D50/D51/D52	leave physically open; no waveform is generated
Reference	GND	named reference for a meter on D12

At a conservative 2 V LED drop from a measured 5 V rail, a 1 k Ω resistor bounds the external indicator near 3 mA. Remove USB power before wiring or moving probe clips. Stop on heat, an unstable USB connection, unexpected illumination, or disagreement between the table and breadboard.



Alternative text: graphite orientation plate showing a Mega 2560, a 10 kilohm potentiometer whose wiper is TP-M/A0, activity and fault evidence, and a probe at logical D49. Above it, the sample joins simulated clock state in an 11-byte record; abstract I2C and SPI owners feed that record, append stages it, sync advances a fixed in-memory durable prefix,

and restart discards an unsynchronized suffix. No soil probe, RTC module, storage card, peripheral, or physical bus wiring is present. The table above is electrically authoritative.

3 From voltage to a valid sample

The reference calibration is:

```
const adk::MoistureCalibration moistureCalibration = {200, 800, 32};
```

A raw reading of 200 represents the chosen dry endpoint and maps to 0 permille. A raw reading of 800 represents the chosen wet endpoint and maps to 1000 permille. Values within 32 counts beyond either endpoint are clamped; larger excursions become `InputBelowRange` or `InputAboveRange`. The names describe electrical range direction, not a diagnosis of soil.

State	Numeric field	Meaning
Unavailable	not usable	no accepted update or sensor shut down
Valid	0–1000 permille	calibrated reading accepted
InputBelowRange	zero	reading below the guarded electrical range
InputAboveRange	zero	reading above the guarded electrical range
Stale	retained value	last valid observation is older than policy

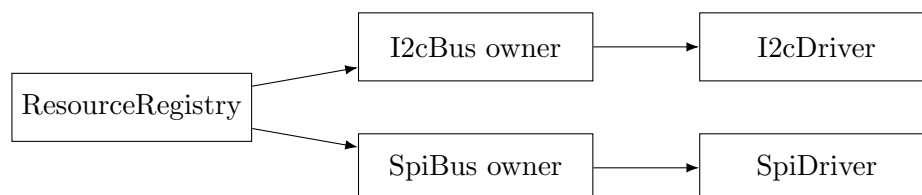
The potentiometer is a repeatable voltage source, not a soil model. ADC counts are ratios to the measured reference rail. Record TP-M relative to GND and do not call an unmeasured count a voltage. A future soil probe needs exact module identity, supply and output limits, source impedance, common ground, corrosion policy, and a separate bench card.

3.1 Integer mapping

$$m = \text{round} \left(\frac{(\text{raw} - \text{dry})1000}{\text{wet} - \text{dry}} \right).$$

The implementation uses signed comparisons around the guarded endpoints, integer-only nearest rounding, and clamping after validity checks. Ascending and descending calibration both work. Calling `sample(now, staleAfter)` does not mutate the stored result; repeated observation is idempotent.

4 Ownership before transport



Construction is inert. `initialize()` claims the named bus resource, then configures its driver. If configuration fails, the owner disables the driver and releases the claim. Repeated initialization is idempotent. `shutdown()` disables transport, releases the claim, and may be called again. Destruction calls shutdown, making cleanup safe during a caller's exception-driven unwinding even though ADK throws no exceptions.

4.1 Why the driver is injected

The same owner can use a deterministic host fake, this lesson's recording driver, or the direct Mega adapter. The owner controls lifetime and request validation. The driver controls platform effects. Neither owns application meaning such as “read a clock” or “append a record.”

The recording drivers deliberately return computed bytes. That makes repeated runs observable to code without pretending the bytes came from hardware.

5 Request boundaries

Owner	Current validation
I2cBus	initialized; address 0x08–0x77; at least one direction; each buffer at most 32 bytes; nonzero sizes require non-null buffers
SpiBus	initialized; nonzero clock; mode 0–3; 1–256 bytes; at least one buffer; chip-select pin must be claimable

The caller retains every buffer and may reuse or discard it after `transfer()` returns. The bus stores no callback and starts no hidden queue. Every call includes a nonzero `Duration` timeout. A physical driver must honor that bound: “synchronous” must not mean “wait forever.”

5.1 Status is policy input

```
const adk::Status status =
    metadataDevice.transfer (writeData, writeSize, readData, readSize,
                             adk::Duration (20));

if (!status.ok ())
{
    observeFailure (status);
}
```

Pass the complete `Status` to the observation boundary. Use `error()` only when a named policy needs the underlying `StatusCode`. `transient()` classifies a potentially recoverable cause; it is not permission to retry blindly. A retry policy belongs above the bus and must be bounded and observable.

6 Clock state and durability

`Rtc::read()` returns `Result<ClockReading>`. Transport status and clock state answer different questions. A successful transport may still report `NotSet` or `OscillatorStopped`; neither is acceptable as a trustworthy record timestamp.

Clock state	Record policy
Valid	serialize raw Unix seconds; civil-time formatting stays out
NotSet	reject timestamp and show fault
OscillatorStopped	reject timestamp and preserve diagnostic state
TransportFault	propagate complete failure status

The sketch uses a clearly named `SimulatedRtc`: it starts at a fixed Unix-seconds value and advances deterministically on reads. It is not an I2C RTC adapter and survives no power interruption.

`Storage::append()` stages bytes. It does not claim durability. `Storage::sync()` is the commit boundary. `FixedStorageMedium` holds a bounded 512-byte in-memory medium while `FixedStorage` owns one session. Shutting down and reinitializing the owner discards any unsynchronized suffix and resumes at the exact durable prefix. A failed sync leaves the staged suffix available for a bounded retry without re-appending; restarting instead discards it.



The record layout is fixed and endian-explicit:

Bytes	Field	Meaning
0–3	Unix seconds	unsigned, most-significant byte first
4	clock state	explicit <code>ClockState</code>
5–6	raw ADC	unsigned, most-significant byte first
7–8	moisture	permille, most-significant byte first
9	sample state	explicit <code>MoistureSampleState</code>
10	metadata token	deterministic I2C recording-driver response

Every eighth committed record, the sketch restarts the `FixedStorage` owner and compares its staged size with the medium’s durable size. This proves session restart against the in-memory model, not board reset or power-loss retention. At fixed capacity, the next append fails visibly rather than overwriting an earlier record.

7 Narrative sketch

```

void loop ()
{
    const adk::TimePoint now (millis ());

    clearAcknowledgement (now);

    if (sampleIsDue (now))
    {
        adk::MoistureSample sample;

        observeMoisture      (now, sample);
        persistMoisture      (now, sample);
        acknowledgeMoisture  (now, sample);
    }
}

```

The canonical example names the high-level actions before recording mechanics. Setup acquires the evidence LEDs, bus and device owners, moisture sensor, simulated RTC, and storage session. Each scheduled cycle observes TP-M, rejects a non-valid sample or clock, obtains one deterministic I2C metadata token, sends the 11-byte record through the SPI recording driver, appends and synchronizes the same bytes, then pulses D13.

The implementation uses elapsed unsigned time and no blocking `delay()`. A recorded input trace produces the same samples, records, statuses, and pulse durations on every replay.

8 Experiment A: calibrate TP-M

Predict. At raw 200, the sample is valid at 0 permille and D13 pulses for about 100 ms. At raw 500, it is near 500 permille and the pulse is near 500 ms. At raw 800, it is 1000 permille and the pulse is near 900 ms.

Observe. With USB removed, place meter clips between TP-M and GND. Restore USB, turn the shaft, and record raw count, TP-M, sample state, permille, and D13 pulse duration. Serial may reveal the count during a diagnostic continuation but is not required to see TP-M or the pulse.

Interpret. Increasing pulse duration supports the calibrated mapping. TP-M supports the electrical input. D13 supports accepted interpretation and record-path completion. It does not prove soil moisture.

Raw target	TP-M	State	Permille	D13 duration
200	_____	_____	_____	_____
500	_____	_____	_____	_____
800	_____	_____	_____	_____

9 Experiment B: predict ownership

Predict. With D12 already claimed by another component, `faultEvidence.initialize()` fails before the two bus owners are started. With all resources free, both owners initialize.

Observe. Use the deterministic host tests for claim conflicts. On the Mega example, observe whether the normal transaction acknowledgement begins.

Interpret. A missing pulse can reject the hypothesis that the whole path completed. A pulse supports the software path only. The landed Mega drivers own D20/D21 for I2C and D50–D53 for SPI, but this recording-driver experiment does not exercise those pins.

Condition	Prediction	Observation	Interpretation
Resources free	_____	_____	_____
Bus busy	_____	_____	_____
Driver failure	_____	_____	_____

10 Experiment C: inspect a transaction

Predict. Every successful cycle records I2C address 0x50, one written byte, and one synthetic metadata token. SPI then records mode 0, 1 MHz, most-significant-bit first, D49 as the logical chip-select identifier, and 11 bytes. Append advances only the staged size; sync advances the durable size.

Observe. Run the host test and inspect the recording driver's exact fields. Run it twice from a clean fixture.

Interpret. Equal records support deterministic request construction. They do not establish I2C acknowledgement, SPI edge timing, voltage compatibility, or chip-select restoration on a Mega.

Recorded fact	First replay	Second replay
I2C address and sizes	_____	_____
SPI settings and size	_____	_____
Exact 11-byte record	_____	_____
Ordered status results	_____	_____

11 Experiment D: distinguish two proofs

Resource proof. A successful first transaction supports that both bus owners and persistent D49 device claim were available to this software path.

Safe-state proof. Host tests show the landed Mega adapters restore their bus pins and chip select to documented inactive states after shutdown and injected failure. The recording driver has no physical pins, so this lesson cannot turn that trace into a measurement. D12 and D13 can demonstrate their own LED endpoint cleanup only.

Question	Available evidence	Still open
Did software acquire the owners?	status and completed D13 pulse	conflict and rollback replay
Did I2C move electrically?	none	SDA/SCL logic-analyzer trace
Did SPI restore chip select?	none	TP-CS transition and inactive level
Did LED cleanup become safe?	endpoint shutdown trace	Mega pin measurement

12 Fault laboratory

Work through these cases with recording fakes before attaching a peripheral:

1. reject I2C reserved addresses 0x00 and 0x78;
2. reject null data with a nonzero size;
3. accept the exact maximum sizes and reject one byte beyond;
4. inject driver-configuration failure and prove claim rollback;
5. inject transfer failure and preserve its complete **Status**;
6. conflict on D49 and prove the SPI driver is not called;
7. inject append failure at every byte and retain only the exact prefix;
8. fail sync, restart the owner, and discard the unsynchronized suffix;
9. fill the fixed capacity and reject the next record without overwrite;
10. replay every **ClockState** separately from transport status;
11. call shutdown twice and then reuse each released resource;
12. destroy initialized owners and prove the same reuse.

Design question. `SpiDevice` owns D49 and asks its driver to configure and release that chip select. What recording operations must the Mega driver expose so tests prove “assert, transfer, restore” across success, partial failure, shutdown, and destruction?

13 Physical adapter acceptance card

This card stays blank until the landed Mega adapters are exercised with exact devices and the named electrical observations are recorded.

Field	Record
Date, revision, commit	
Mega board and USB supply	
Measured 5 V reference and TP-M range	
Potentiometer marking and endpoint counts	
Exact I2C specimen, voltage, address	
Exact SPI specimen, voltage, CS polarity	
Logic analyzer and firmware tool versions	
SDA/SCL prediction and observation	
MOSI/MISO/SCK/CS prediction and observation	
Timeout/fault injection	
Shutdown and reset safe state	
Reviewer and interpretation	
☐ Wiring was inspected with USB removed. ☐ TP-M remained within GND and the measured reference rail. ☐ Dry/wet endpoints and guarded faults matched the prediction. ☐ Device logic voltage and common ground match primary data. ☐ Acquisition and electrical safe state were proved separately. ☐ No back-power path exists when either side is unpowered. ☐ The transaction has a bounded completion or timeout. ☐ SPI chip select is inactive after success, failure, and reset. ☐ Observations, deviations, and stop conditions are recorded.	

14 Exercises

1. Reverse the dry and wet calibration endpoints. Predict the three sample values and pulse durations before replaying them.
2. Draw the owner, driver, device, and application-policy lifetimes. Mark every borrowed relationship.
3. Add a fake-driver failure at the first byte and state which facts may still be claimed.
4. Propose a timeout value object without reading a global clock inside the pure bus owner.
5. Design a non-owning I2C device relationship that cannot outlive its owner. Compare generation tokens, scoped transactions, and explicit registration.
6. Write the SPI restoration invariant for success, invalid input, resource conflict, partial transport failure, shutdown, and destruction.

15 CLI record

```
make host-test
make arduino-Lesson022OwnedBuses
make upload-Lesson022OwnedBuses PORT=/dev/ttyACM0
make hardware-card LESSON=022 BOARD=mega2560
make evidence LESSON=022 SIGNAL=D13
make lessons
make lessons-check
make site-check
```

The lesson-specific Make targets become release claims only after the integrator registers the package. A successful compile remains firmware evidence, not physical bus evidence.

16 References and next work

- Arduino Mega 2560 Rev3 documentation and pinout: docs.arduino.cc.
- NXP I2C-bus specification and user manual UM10204: [nxp.com](https://www.nxp.com).
- Microchip ATmega640/1280/1281/2560/2561 datasheet: [microchip.com](https://www.microchip.com).

Next, select exact RTC and storage specimens and run the landed Mega adapters through the blank electrical acceptance card. Host restoration tests remain software evidence until those observations are recorded. Lesson 023 consumes the pure foundation through inert, visibly bounded load intent.